Campus Smart-Parking System

Object-Oriented Design and Java Implementation (Tasks 1–12)

Name: Sanjay Adithya S J | **Register No:** 192311336 | **Course:** CSA0921 – Programming in Java |

Contents

1. Understand the Problem and Identify Objects
2. Choose the Data Representation
3. Design the Control Logic
4. Design Classes, Objects and Constructors
5. Static Members and the Object Life Cycle
6. Design the Inheritance Hierarchy
7. Override and Specialise Behaviour
8. Apply Polymorphism
9. Introduce Abstraction
10. Encapsulation through Packages and Access Modifiers
11. Implement, Test and Defend the Design
12. Re-evaluate and Adapt the Design

Task 1 – Understand the Problem and Identify the Objects

Problem. The Campus Smart-Parking System manages parking for students, staff and visitors. For every arriving vehicle it allocates a suitable free slot, issues a unique ticket, records entry and exit times, calculates the fee and frees the slot on exit. It supports two-wheelers, cars and electric vehicles (EVs). EV charging is billed separately. Students and staff with a valid monthly pass park free; visitors pay the normal fee.

Inputs and outputs

Inputs	Outputs
Registration number, vehicle type, floor/slot availability, entry and exit times, pass validity, EV charging need, parking and charging rates	Allocated slot, unique ticket number, entry/exit record, parking fee, charging fee, updated availability

Candidate classes and responsibilities

Class	Responsibility
Vehicle	Holds common vehicle data and shared behaviour
TwoWheeler / Car / ElectricVehicle	Category-specific properties and rates
ParkingSlot	Floor/slot number, slot type, occupancy; prevents double allocation
ParkingFloor	Manages the slots of one floor
ParkingLot	Coordinates floors, allocation, entry and exit
ParkingTicket	Unique ticket number, vehicle, entry and exit time
ParkingPass	Checks whether a monthly pass is valid
FeeCalculator	Computes parking fee and separate charging fee

Verbs that become methods

Verb in the problem	Method
Enter a vehicle	parkVehicle()
Allocate a slot	allocateSlot()
Issue a ticket	generateTicket()
Record entry / exit	recordEntryTime() / recordExitTime()
Calculate a fee	calculateFee()
Release a slot	releaseSlot()
Validate a pass	validatePass()

Verb in the problem	Method
Find a vehicle	findVehicle()
Check availability	isAvailable()

Business rules

- A slot holds at most one vehicle at a time.
- Ticket numbers are unique and never reused.
- A vehicle enters only if a suitable slot is free; an EV needing charge must get a charging-capable slot.
- Fee depends on vehicle type and duration; charging is billed separately.
- A valid student/staff monthly pass waives parking; visitors pay.
- On exit the slot is released; invalid or already-closed tickets cannot be used.
- Duration can never be negative; floors may have different capacities.

Assumptions: registration numbers are unique; slot numbers are unique within a floor; rates are set by the facility; pass validity is checked at entry; charging fees apply only when charging is used.

Objects vs parallel arrays

Feature	Object-oriented design	Parallel arrays
Storage	Related fields grouped in one Vehicle object	Separate arrays for registration, type, time
Search by registration	O(n) linear search	O(n) linear search in regNo[]
Adding an attribute	Add one field	Add another array and keep indexes in sync
Maintainability	Supports inheritance and polymorphism	Gets harder as fields grow
Space	O(n) plus object overhead	O(n) across arrays

Conclusion: the object-oriented design is chosen because it keeps related data together and is easy to extend with new vehicle types, rules and pricing.

Task 2 – Choose the Data Representation

Java data types

Field	Type	Reason
Floor / slot number	int	Whole-number indexes
Registration number	String	Letters, digits and symbols
Vehicle type	enum or String	Category label
Entry / exit time	long	Millisecond timestamps
Duration	long	Elapsed minutes or milliseconds
Fee	BigDecimal (preferred)	Avoids floating-point rounding errors
Slot availability	boolean, or null occupancy	Free/occupied state
Ticket number	long	Numeric unique identifier

Floors and slots. Floors have different capacities, so a jagged 2-D array is used. `null` means free; a registration number means occupied.

```
String[][] slots = new String[3][];
slots[0] = new String[20];
slots[1] = new String[50];
slots[2] = new String[100];
```

Type conversion and integer-division risk. Integer division drops the fractional part, so 90 minutes / 60 gives 1 hour, which under-bills. Adding 59 before dividing rounds up to the next started hour.

```
public class FeeDemo {
    public static void main(String[] args) {
        long minutes = 90;
        long wrongHours = minutes / 60;            // 1 (incorrect)
        long billableHours = (minutes + 59) / 60;  // 2 (correct)
        double fee = billableHours * 40.0;
        System.out.println(wrongHours + " " + billableHours + " " + fee);
    }
}
```

```
Output: 1 2 80.0
```

For real money, use BigDecimal or whole paise instead of double.

Reading input

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter vehicle number: ");
String vehicleNumber = sc.nextLine();
System.out.print("Enter vehicle type: ");
String vehicleType = sc.nextLine();
System.out.println(vehicleNumber + " " + vehicleType);
sc.close();
```

Alternative representations

Representation	Access	Time (known slot)	Space
Jagged array	slots[floor][slot]	O(1)	O(S)
1-D array	offset[floor] + slot	O(1)	O(S)
ArrayList of String[]	slots.get(floor)[slot]	O(1)	O(S) plus list overhead

The formula floor × slotsPerFloor + slot only works for equal floor sizes. For unequal floors use prefix offsets or, more simply, a jagged array.

Task 3 – Design the Control Logic

Pseudocode – allocate a free slot

```
ALGORITHM AllocateSlot(vehicleNo)
  FOR each floor f
    FOR each slot s on floor f
      IF slots[f][s] is null THEN
        slots[f][s] <- vehicleNo
        RETURN {f, s}
  RETURN null            // facility full
```

Pseudocode – parking fee (first started hour ₹40, each further started hour ₹20, daily cap ₹200)

```
ALGORITHM CalculateFee(durationMinutes)
  IF durationMinutes <= 0 THEN RETURN 0
  hours <- CEILING(durationMinutes / 60)
  fee   <- 40 + (hours - 1) * 20
  IF fee > 200 THEN fee <- 200
  RETURN fee
```

Java implementation

```
public class ParkingFacility {
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++)
            slots[f] = new String[slotsPerFloor[f]];
    }

    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++)
            for (int s = 0; s < slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
        return null;
    }

    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});

System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
```

```
System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));

System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));

System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " " +
p.release(5, 0));

System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

```
Output:
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Control statements. `return` exits as soon as a free slot is found; `break` stops a loop early; `continue` skips an unsuitable slot. A `null` result means the facility is full and the caller must report it.

Approach	Allocation	Release	Extra management
Scan all slots	O(F × S) worst case	O(1) with indexes	Simple; O(1) extra space
Free-slot stack per floor	O(1) pop	O(1) push	Stores up to O(F × S) slot references; must avoid duplicate entries

Task 4 – Design Classes, Objects and Constructors

Class	Example fields	Example methods
Vehicle	vehicleNo, vehicleType	getVehicleNo(), getVehicleType()

Class	Example fields	Example methods
ParkingSlot	floorNo, slotNo, vehicleNo	isAvailable(), parkVehicle(), releaseVehicle()
ParkingFloor	floorNo, slots	findFreeSlot(), displaySlots()
ParkingTicket	ticketNo, vehicle, entryTime, exitTime	recordExit(), getDuration()
ParkingPass	passNo, expiryDate	isValid()
FeeCalculator	hourlyRate, dailyMaximum	calculateFee()
ParkingFacility	floors, tickets	allocate(), release(), generateTicket()

Constructors, `this` and constructor chaining

```
class Vehicle {
    String vehicleNo;
    String vehicleType;

    Vehicle() { this("UNKNOWN", "Car"); }
    Vehicle(String vehicleNo, String vehicleType) {
        this.vehicleNo = vehicleNo;
        this.vehicleType = vehicleType;
    }
    Vehicle(String vehicleNo) { this(vehicleNo, "Car"); }
    Vehicle(Vehicle other) { this(other.vehicleNo, other.vehicleType); } //
copy constructor
}

class ParkingTicket {
    long ticketNo;
    Vehicle vehicle;
    long entryTime;
    long exitTime;

    ParkingTicket(long ticketNo, Vehicle vehicle, long entryTime) {
        this.ticketNo = ticketNo;
        this.vehicle = vehicle;
        this.entryTime = entryTime;
        this.exitTime = 0;
    }
```

```
    void recordExit(long exitTime) { this.exitTime = exitTime; }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Vehicle("TN01AB1111", "Car");
        Vehicle v2 = new Vehicle("TN02CD2222");
        ParkingTicket t1 = new ParkingTicket(1001, v1,
System.currentTimeMillis());
        System.out.println(v1.vehicleNo);
        System.out.println(v2.vehicleType);
        System.out.println(t1.ticketNo);
    }
}
```

```
Output:
TN01AB1111
Car
1001
```

Notes. A default constructor builds a basic object; overloaded constructors give different initialisation options. `this.field` separates a field from a parameter, and `this(...)` calls another constructor of the same class and must be the first statement. A copy constructor may copy vehicle details but must never copy a ticket number, because ticket numbers must stay unique. Reference variables live in stack frames, objects on the heap, and an object becomes eligible for garbage collection once it is unreachable.

Alternative creation approaches

Approach	Validity and trade-off
Parameterised / overloaded constructor	Object is complete immediately; O(1)
Default constructor + setters	Object may be incomplete or invalid between calls; O(1)
Static factory method	Can validate input and control creation; O(1) for fixed validation

Creating n tickets costs O(n) time and O(n) space with any of the three.

Task 5 – Apply Static Members and the Object Life Cycle

```
class ParkingTicket {
    private static long ticketCounter = 1000;
    private final long ticketNo;
    private final String vehicleNo;

    public ParkingTicket(String vehicleNo) {
        ticketNo = ++ticketCounter;
        this.vehicleNo = vehicleNo;
    }
    public long getTicketNo() { return ticketNo; }
    public static long getTicketCounter() { return ticketCounter; }
}
```

```
Sample result after issuing 3 tickets:
Ticket numbers: 1001, 1002, 1003
Tickets issued: 3
```

`ticketCounter` belongs to the class and is shared by all tickets, which guarantees uniqueness. As an instance field every ticket would get its own counter and duplicates would appear. A static block suits one-time initialisation, and static methods work on class-level data without an object.

Object life cycle

- A ticket is created at entry with its number, vehicle and entry time.
- At exit the system records the exit time, calculates the fee and releases the slot.
- The ticket is garbage-collectable once nothing reachable refers to it.
- Methods can take objects as parameters (calculateFee(ParkingTicket)) and return them (generateTicket()).

Counter approach	Space for k facilities	Uniqueness
One static counter	O(1)	Unique across facilities if shared safely
Instance counter	O(k)	Unique only within a facility unless combined with a facility ID

Ticket storage is O(n) either way. Concurrent entry gates would need an atomic counter or database-generated IDs.

Task 6 – Design the Inheritance Hierarchy

```
          Vehicle
          /     \
   TwoWheeler   Car
                 |
            ElectricCar
```

TwoWheeler and Car extend Vehicle (hierarchical); ElectricCar extends Car (multilevel). Vehicle holds common registration data, Car adds seats, and ElectricCar adds battery and charging details.

```
class Vehicle {
    String vehicleNo;
    Vehicle(String vehicleNo) {
        this.vehicleNo = vehicleNo;
        System.out.println("Vehicle constructor");
    }
}
class Car extends Vehicle {
    int seats;
    Car(String vehicleNo, int seats) {
        super(vehicleNo);
        this.seats = seats;
        System.out.println("Car constructor");
    }
}
class ElectricCar extends Car {
    int batteryCapacity;
    ElectricCar(String vehicleNo, int seats, int batteryCapacity) {
        super(vehicleNo, seats);
        this.batteryCapacity = batteryCapacity;
        System.out.println("ElectricCar constructor");
    }
}
public class Main {
```

```
    public static void main(String[] args) {
        new ElectricCar("TN01AB1111", 5, 60);
    }
}
```

```
Output:
Vehicle constructor
Car constructor
ElectricCar constructor
```

Constructors run from the top superclass down. `super(...)` calls the parent constructor and must be the first statement.

Feature	Inheritance	Composition
Model	ElectricCar is-a Car	Vehicle has-a VehicleType
Adding categories	Usually add a subclass	Add a new type/strategy object
Changing pricing at runtime	Less flexible	Strategy can be swapped
Fee calculation	O(1)	O(1)
Space	Fields plus object overhead	Fields plus a type reference (shareable)

Task 7 – Override and Specialise Behaviour

Subclasses override a superclass method to give category-specific behaviour. Here each vehicle overrides calculateFee(); Vehicle also overrides toString(), equals() and hashCode().

```
class Vehicle {
    protected final String vehicleNo;
    Vehicle(String vehicleNo) { this.vehicleNo = vehicleNo; }
    public double calculateFee(int hours) { return hours * 40.0; }
    @Override public String toString() { return "Vehicle Number: " +
vehicleNo; }
    @Override public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Vehicle)) return false;
```

```
        return vehicleNo.equals(((Vehicle) obj).vehicleNo);
    }
    @Override public int hashCode() { return vehicleNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String n) { super(n); }
    @Override public double calculateFee(int hours) { return hours * 20.0; }
}
class Car extends Vehicle {
    Car(String n) { super(n); }
    @Override public double calculateFee(int hours) { return hours * 40.0; }
}
class ElectricCar extends Car {
    ElectricCar(String n) { super(n); }
    @Override public double calculateFee(int hours) { return hours * 40.0 +
30.0; }
}
public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new TwoWheeler("TN01AB1111");
        Vehicle v2 = new Car("TN02CD2222");
        Vehicle v3 = new ElectricCar("TN03EF3333");
        System.out.println(v1.calculateFee(2));
        System.out.println(v2.calculateFee(2));
        System.out.println(v3.calculateFee(2));
        System.out.println(v1);
        System.out.println(v1.equals(new TwoWheeler("TN01AB1111")));
    }
}
```

```
Output:
40.0
80.0
110.0
Vehicle Number: TN01AB1111
true
```

Overriding rules. @Override makes the compiler check the override. Name and parameter types must match; the return type must be identical or covariant; visibility cannot be reduced. A final method cannot be overridden, a final class cannot be extended, and a final field cannot be reassigned. If two objects are equal their hash codes must be equal.

Feature	Overriding	Switch on type
Dispatch	O(1)	Up to O(k) checks for k types
Adding a type	Add a subclass	Edit the switch
Maintenance	Behaviour kept per class	Logic grows and centralises

Task 8 – Apply Polymorphism

```
import java.time.Duration;
import java.time.LocalTime;

interface Chargeable { double chargingFee(int kWh); }

abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {            // overload 1
        if (minutes <= 0) return 0;
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
    public double calculateFee(LocalTime in, LocalTime out) { // overload 2
        long minutes = Duration.between(in, out).toMinutes();
        if (minutes < 0) minutes += 24 * 60;
        return calculateFee((int) minutes);
    }
    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String r) { super(r); }
    @Override protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
```

```
    Car(String r) { super(r); }
    @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r) { super(r); }
    @Override public double chargingFee(int kWh) { return kWh * 12.0; }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"), new
ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable) fee += ((Chargeable)
v).chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " +
v.getRegNo() + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new
ElectricCar("TN02B2")));
    }
}
```

```
Output:
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Dispatch trace

- **Compile-time polymorphism:** calculateFee(int) and calculateFee(LocalTime, LocalTime) are overloads; the compiler picks one from the arguments.
- **Runtime polymorphism:** the Vehicle[] array holds different subclasses, and the JVM calls the ratePerHour() of the actual object.
- **ElectricCar:** 130 minutes rounds up to 3 hours, so parking = 3 × ₹30 = ₹90 and charging = 10 × ₹12 = ₹120, total ₹210.
- The last line is true because equals() compares only registration numbers, which is valid only if the registration alone defines identity.

Alternative: a chain of instanceof checks costs O(k) and must be edited for every new type; dynamic dispatch is O(1) and needs no change to existing code.

Task 9 – Introduce Abstraction

Vehicle is abstract because the system only creates specific vehicle types, and ratePerHour() is abstract because each type has its own rate. Chargeable adds an optional charging capability and Payable describes the amount to be paid.

```
abstract class Vehicle {
    protected final String regNo;
    Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    public double calculateFee(int hours) { return hours * ratePerHour(); }
}
interface Chargeable {
    double chargingFee(int kWh);
    default void showChargingInfo() { System.out.println("Vehicle supports
charging."); }
    static double standardRate() { return 12.0; }
}
interface Payable { double paymentAmount(); }

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
}
class ElectricCar extends Car implements Chargeable, Payable {
    private final int kWh;
    ElectricCar(String regNo, int kWh) { super(regNo); this.kWh = kWh; }
    @Override public double chargingFee(int kWh) { return kWh *
Chargeable.standardRate(); }
    @Override public double paymentAmount() { return calculateFee(2) +
chargingFee(kWh); }
}
public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar("TN03C3", 10);
        System.out.println("Parking Fee: Rs " + e.calculateFee(2));
        System.out.println("Charging Fee: Rs " + e.chargingFee(10));
```

```
        System.out.println("Total Fee: Rs " + e.paymentAmount());
        e.showChargingInfo();
    }
}
```

```
Output:
Parking Fee: Rs 60.0
Charging Fee: Rs 120.0
Total Fee: Rs 180.0
Vehicle supports charging.
```

Feature	Abstract class Vehicle	Interface Chargeable
Purpose	Shared state and implementation	Capability that selected classes add
Fields	Instance fields allowed	Only public static final constants
Constructors	Allowed	Not allowed
Methods	Abstract and concrete	Abstract, default and static
Multiple inheritance	Extend only one class	Implement many interfaces

If Vehicle were an interface and Chargeable an abstract class, Vehicle could not hold per-object state or a constructor, and an EV could not extend any other abstract class while extending Chargeable. Fixed calculations stay O(1) and storage O(n) in both designs, but the reversed design needs duplicated state or helper classes.

Task 10 – Enforce Encapsulation through Packages and Access Modifiers

Package structure

Package	Contents
parking.model	Vehicle, Car, TwoWheeler, ElectricCar, ParkingFloor, ParkingSlot, ParkingTicket, Chargeable, Payable, Pass, OneDayPass

Package	Contents
parking.service	ParkingLot, VehicleFactory, PricingStrategy and its implementations
parking.app	Main entry point
parking.test	Automated tests for normal, boundary and invalid cases

Access modifiers. Fields are private so callers cannot corrupt state; public methods form the API; helper methods stay private; package-private is used only when classes in one package really need it; protected is reserved for subclass behaviour.

Validation examples: registration numbers cannot be blank; floor/slot numbers and capacities must be positive; a ticket needs an entry time; exit cannot precede entry; charging energy cannot be negative; VehicleFactory rejects unknown types.

Getters and setters. A getter exposes a value without allowing change. Identity and ticket data have no setters. Where a setter exists it validates first. Constructor validation is best for fields that must never be invalid.

Wrapper classes and autoboxing. Integer and Double are used in collections and for nullable values. Autoboxing converts int to Integer and unboxing converts back. Do not compare wrappers with `==` (it may compare identity); use equals() or compare primitives after a null check. Unboxing a null wrapper throws NullPointerException.

Why not public fields? With public fields every one of m call sites must repeat the validation, so a check may be duplicated m times and one missed copy can corrupt slot or ticket state. Centralising a simple check in the model is O(1) per update and enforced for every caller.

Conclusion: private fields with validated constructors and methods protect invariants and remove duplicated checks. The cost is some extra boilerplate, justified for a system that handles real capacity and billing data.

Task 11 – Implement, Test and Defend the Design

Implementation. The project is split into model, service, app and test packages. It allocates a suitable slot, creates a unique ticket, records entry time, releases the slot on exit and computes parking and charging fees separately.

Test evidence (12 passed, 0 failed)

- Normal: different vehicle types enter and leave.
- Boundary: last free slot, zero-minute stay, exact fee-slab boundary.
- Invalid: unknown vehicle type, duplicate active registration, invalid floor/slot configuration, invalid exit time, charging a non-EV.
- Pass cases: a monthly pass waives parking; EV charging is billed separately; an EV gets a charging-compatible slot.
- The full output is in the project archive as test-results.txt.

Manual daily collection check (Task 8–9 rates)

Vehicle	Calculation	Amount
Car, 2 started hours	2 × ₹30	₹60
Two-wheeler, 1 started hour	1 × ₹10	₹10
EV parking, 2 started hours	2 × ₹30	₹60
EV charging, 10 kWh	10 × ₹12	₹120
Total		**₹250**

The program output for this scenario should also be ₹250.

Array vs ArrayList vs HashMap

Structure	Lookup by registration	Notes
Array	O(n) scan	Compact but fixed capacity
ArrayList	O(n) scan	Resizable
HashMap<String, ParkingTicket>	Expected O(1)	Insert/remove expected O(1); O(n) space for n active tickets

Slot allocation is still O(S) because it scans S total slots.

Evaluation. Encapsulation protects fields, inheritance shares vehicle state, polymorphism lets ParkingLot accept any Vehicle subtype, and abstraction (Chargeable, Payable, PricingStrategy) separates capabilities and policies. **Limitation:** the switch-based factory must be edited for every new vehicle category; a registry-based factory would remove that.

Task 12 – Re-evaluate and Adapt the Design

A. Impact analysis. New requirements: e-bikes with a flat fee, delivery vans at twice the car rate from 9 AM–5 PM, a one-day visitor pass, and five floors of different sizes. New classes: ElectricBike, DeliveryVan, Pass, OneDayPass. Pricing changes are isolated in AdaptedPricing, and ParkingLot is extended to attach and validate a day pass. A rectangular ParkingSlot[5][100] would waste or invalidate positions, so each ParkingFloor owns its own slot array. The sample configuration {20, 35, 50, 75, 100} has 280 slots.

B. Evaluation of the original design. Modify, do not redesign. The Vehicle hierarchy, ticket life cycle, floor/slot model, active-ticket HashMap and PricingStrategy all remain useful. The factory still needs one new entry per vehicle type, but new pricing can be added without rewriting old pricing. Pass validity sits in a separate Pass abstraction, not in each vehicle subclass.

C. Adapted structure and pricing

- ElectricBike extends the two-wheeler model and implements Chargeable; DeliveryVan extends Vehicle; OneDayPass implements Pass and is valid for one calendar date.
- Car ₹30 per started hour; two-wheeler ₹10 per started hour; e-bike ₹15 flat per session; charging ₹12/kWh.
- Delivery van ₹60 per started hour if the stay overlaps 9 AM–5 PM, otherwise ₹30.
- A valid day pass waives parking when entry and exit dates both match the pass date; charging stays separate.
- The e-bike fee and pass benefit are not given in the brief, so they are stated as illustrative assumptions.
- The van rule doubles the rate for the whole session if any part overlaps 9 AM–5 PM. If only the daytime portion should be doubled, the strategy must split the stay into daytime and off-hours intervals.

D. Evidence-based defence

Test	Result
E-bike flat parking fee	PASS
Delivery van daytime double rate	PASS
Delivery van outside business hours uses car rate	PASS
One-day pass waives parking fee on valid date	PASS

Test	Result
Unequal capacities across five floors	PASS
Unknown type rejected	PASS

Task 12: 6 passed, 0 failed. With the original 12 tests, the project has 18 passing tests, which supports modifying the existing design rather than replacing it.

Complexity: slot allocation O(S) worst case; HashMap ticket lookup expected O(1); storage O(S + n) for n active tickets.

Final Recommendation

Keep and modify the original architecture. New categories and the visitor pass are added through new classes, and the revised pricing sits behind PricingStrategy. Remaining production work: real pass-purchase accounting, precise daytime-only van billing if required, persistent storage, replacing factory conditionals with a registry, and concurrency protection.

Overall Conclusion

The Campus Smart-Parking System is best modelled with object-oriented principles: suitable data types and jagged arrays hold facility data; loops and validation control allocation and release; constructors create valid objects; static members manage ticket numbering; inheritance and overriding specialise vehicle behaviour; polymorphism removes type-specific conditionals; abstraction separates common state from optional capabilities such as EV charging; and encapsulation keeps the design maintainable as rules change.